Final Project: Marker-based Augmented Reality

Course: Basic Computer Vision | Student: Wang Zhaofeng | Date: 17/10/2026

The software and GUI were tested on synthetic images and on a phone-displayed marker through the integrated webcam.

1. Method and implementation

The program finds a reference image in a video frame, then replaces it with another image. Both reference graphics were generated for this project. The application accepts a webcam or a recorded video.

Step	Choice and reason
Feature extraction	SIFT is the default. ORB is available for comparison. Marker descriptors are computed once and refreshed when the method or feature limit changes.
Matching	Brute-force kNN with k=2. SIFT uses L2 distance; ORB uses Hamming distance.
Filtering	A ratio test rejects ambiguous matches. Each target feature is used once. RANSAC then selects geometrically consistent pairs.
Alignment	A homography maps the planar marker to the frame. Inlier count, ratio, coverage and the projected outline are checked before replacement.
Augmentation	warpPerspective transforms the substitute image. A warped mask limits blending to the marker region. Failed detection leaves the current frame unchanged.

Default settings: 1800 features; ratio 0.72 for SIFT and 0.78 for the ORB benchmark; RANSAC threshold 3 pixels; at least 10 inliers; inlier ratio at least 0.35; reference-point coverage at least 3%; opacity 1. The GUI lets the user adjust the main values.

The teacher's OpenCV_Image_Utilities.py is used for UTF-8 path reading when available. Its file encoding was converted from cp1252 to UTF-8 without changing the function bodies. Its overlay routine paints a mask with a solid colour, so image replacement is implemented separately.

Optional residual alignment refinement and 3D rendering are not included.

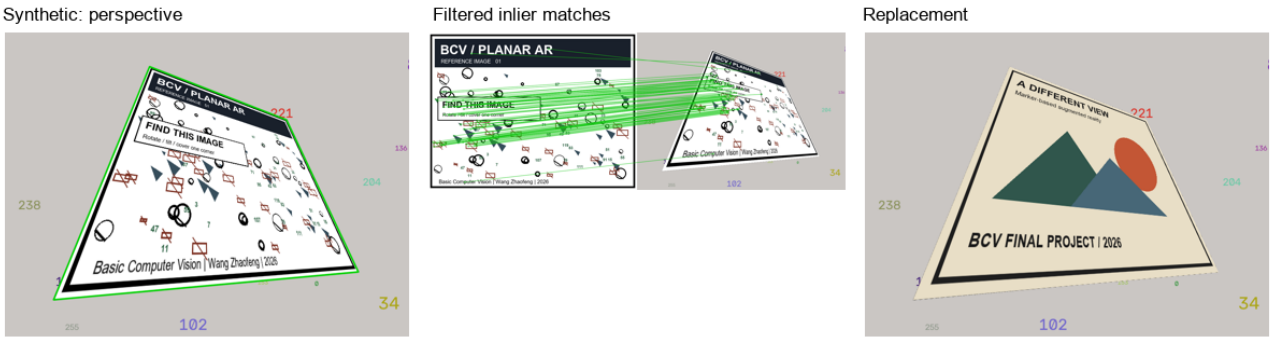
2. Synthetic experiments

The test set contains two marker types, two descriptors and ten conditions, with three small geometric perturbations per condition: 120 runs. Frames are 800 x 600. The known homography provides the true corner locations. A positive run succeeds when the accepted estimate has mean four-corner error at most 5 pixels. Absence cases are counted separately.

Condition	SIFT pass	Error px	ORB pass	Error px
normal	3/3	0.13	3/3	0.62
rotation	3/3	0.20	3/3	0.56
perspective	3/3	0.49	3/3	3.50
dim	3/3	0.13	3/3	0.75
shadow	3/3	0.14	3/3	0.61
blur	3/3	0.56	3/3	2.14
occlusion 25	3/3	0.12	3/3	0.62
occlusion 50	3/3	0.38	3/3	1.38
clutter	3/3	0.14	3/3	0.60

Both methods pass all 27 positive cases for the rich marker. The feature-poor grey rectangle passes none of the 27 positive cases for either method. No marker-absence false positives were observed in the 12 tested negative runs. These are small, controlled synthetic sets, not estimates of real-world reliability.

SIFT gives smaller corner error in the strong perspective case. Performance timings are recorded per run in the CSV files; they exclude debug drawing and GUI display. They should not be read as live camera FPS.



Synthetic perspective test: detected outline, RANSAC inlier matches and replacement.

3. Real phone and webcam tests

The marker was displayed on a phone in front of the integrated camera. An initial frontal test accepted all 203 frames in 12.5 seconds. A second, guided recording covered tilt, rotation, dimming, partial covering, background changes and marker removal. These were handheld actions, not calibrated angle or lighting measurements.

Condition	SIFT accepted	ORB accepted	SIFT ms	ORB ms
frontal	23/23	22/23	159.4	61.7
tilt	20/22	14/22	134.8	45.0
rotation	21/23	8/23	119.1	49.1
dim	3/29	0/29	80.4	30.7
occlusion 25	14/26	0/26	94.6	43.4
occlusion 50	23/23	0/23	105.0	30.6
clutter	19/24	3/24	123.7	45.6
absent	0/21	0/21	96.1	47.1

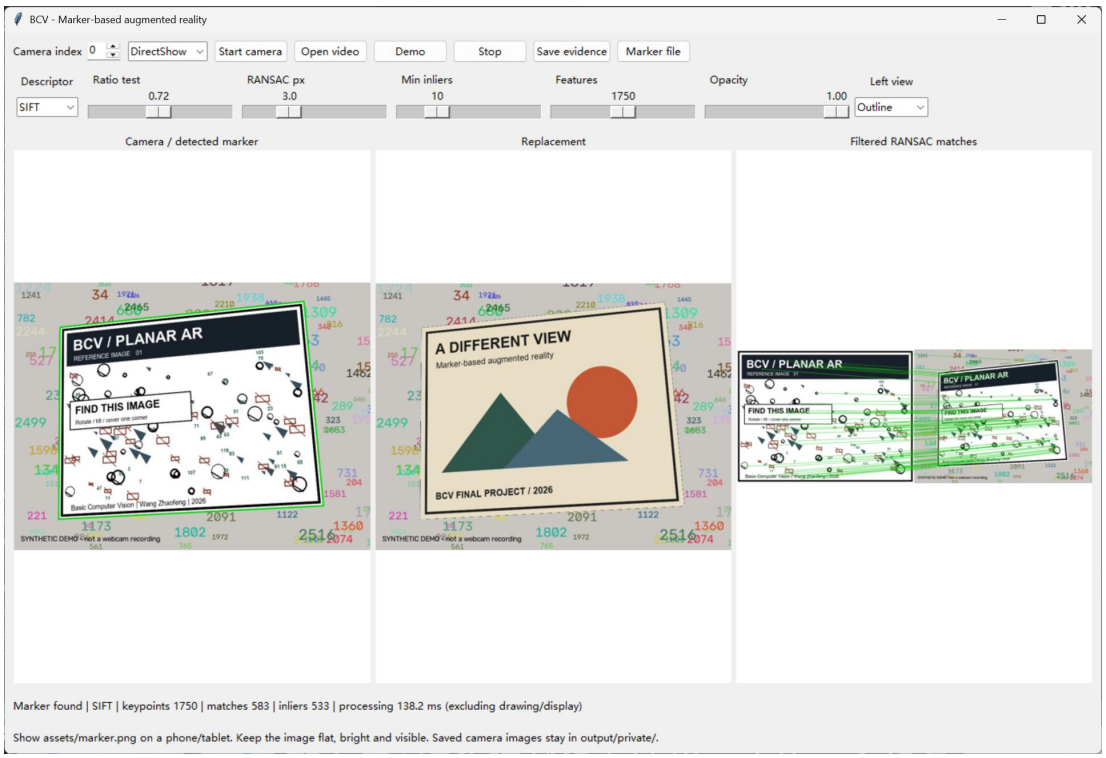
Both methods process the same sampled frames from the MJPG recording, taking every fourth frame. Accepted means the estimate passed the checks; it is not ground-truth accuracy. Timings exclude GUI drawing. SIFT is more reliable in this recording; ORB processes frames faster but loses more difficult views.

Dimming caused frequent losses. The covering steps also included changes in position and focus, so their labels do not establish a measured occlusion percentage. The phone-removal transition before 79 seconds is excluded from the absence row; it still contains the actual marker. Later reviewed samples contain no phone.



Real camera examples and replacements. Pixels outside the estimated marker are whitened for privacy.

4. Interface and limits



Actual application window playing the generated synthetic demo, not a webcam recording.

The GUI provides camera selection, video input, SIFT/ORB selection, matching and RANSAC controls, opacity and two diagnostic views. The replacement and inlier matches appear beside the input frame. Camera capture runs in a separate process so an unavailable virtual camera can be stopped without freezing the interface.

The interface was launched and inspected with a generated demo. A separate guided capture window recorded the real phone tests. The camera input was 640 x 480. Raw camera images stay under output/private/; only marker-masked examples are published.

The application can also save an input frame, outline, replacement, matches and recent metrics with Save evidence. The camera capture process stops when the application closes.

Current limits

The synthetic tests reuse the exact reference artwork and cannot reproduce screen reflections, rolling shutter, focus changes or all camera noise. A plain marker has too few distinctive features. Severe viewpoint changes, blur and occlusion can remove enough matches to prevent alignment.

Occlusion is tested as a detection challenge. The replacement is drawn over the full estimated marker region, including any foreground hand; foreground-aware occlusion handling is not implemented. There is no temporal tracking or smoothing. The guided sequence is a small qualitative test, and no independent real-camera corner accuracy or jitter score is claimed.

References: course project sheet; OpenCV Feature Matching and Feature Matching + Homography tutorials. Tested with Python 3.11.16, OpenCV 5.0.0 and NumPy 2.4.6.